

CS 113

Week 3 Lecture 1



Today's Topics

Homework #2 / Lab #3

Conditional statements

String comparison in Java (a deeper dive)

Methods continued

Homework #1 / #2

Homework #1: due tonight at 11:59pm.

Homework #2: will be released tomorrow (Wed 9/16), due next Tues 9/22 at 11:59pm.

TA Office hours: Mon-Fri 5-7pm in Park 230.

My office hours: Tues 11am-12pm in Park 204.

Lab #3

Two parts:

1. (Actually) useful Vim commands
2. Practice with methods

Decision making: multi-way if statements

```
if (<condition1>) {  
    <stmts>  
} else if (<condition2>) {  
    <stmts>  
}  
....  
else {  
    <stmts>  
}
```

NOTES:

- Conditions evaluated in order
- First true condition executes
- Only **one** of the conditions can execute!
- the final else statement is optional

Example: Height.java

- Write a program (called Height.java) that determines if a user can ride a rollercoaster.
- Make sure to ask the user for height in inches.
- If they are taller than 3, 4, or 5 feet:
 - Print a message saying “you are taller than X feet, you may ride”
- If they are shorter than 3 feet:
 - Print a message saying “you are too short to ride.”

Exercise: Blackjack

Write a program `Blackjack.java` which generates a random value between 2 and 21

- If the value is 21, print the value and “Blackjack” to the console
- If the value is between 17 and 20, print the value and “Stand” to the console
- If the value is less than 17, print the value and “Hit me!” to the console

Comparing strings

In Java, you cannot directly compare strings, we have to use **compareTo**

Comparing strings

- In Java, you cannot directly compare strings: use **compareTo**

```
String a = "apple";
```

```
String b = "banana";
```

```
if (a.compareTo(b) == 0) {
```

```
    System.out.println("a and b match!");
```

```
}
```

```
if (a.compareTo(b) != 0) {
```

```
    System.out.println("a and b DO NOT match!");
```

```
}
```

Lexicographic Values/Order

- Strings are **ordered lexicographically**
 - Generally, the same order as alphabetical order, with some caveats
 - The characters of a string each correspond to a number

ASCII

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com

StringCompare.java

```
String first = "a";
String second = "A";
int asciia = (int) first.charAt(0);
int asciib = (int) second.charAt(0);
System.out.println("ASCII Code for " + first + " is " + asciia);
System.out.println("ASCII Code for " + second + " is " + asciib);

if (first.compareTo(second) == 0) {
    System.out.println(first + " is equal to " + second);
}
else if (first.compareTo(second) < 0) {
    System.out.println(first + " is less than " + second);
}
else if (first.compareTo(second) > 0) {
    System.out.println(first + " is greater than " + second);
}
```

```
$ java StringCompare
ASCII Code for a is 97
ASCII Code for A is 65
a is greater than A
```

Exercise: IsPrimary

Write a program that asks the user for a color and prints whether the color is primary or not.

- The primary colors are “red”, “green”, “blue”
- All other inputs are non-primary

```
$ java IsPrimary  
Enter a color: green  
green is not primary
```

```
$ java IsPrimary  
Enter a color: blue  
blue is primary
```

Let's Try It Out

Demo 1: Ask user for a number, and return the square root

```
Math.sqrt(<number>);
```

Lets round that answer to an integer

Lets now do this for 2 numbers

Lets now do this for 4 numbers

Lets now do this for 6 numbers

Anatomy of a method

- All methods have the following things:
 - Name
 - Parameter
 - Body
 - Return Type

```
public static int method1 (int param1,  
                           String param2) {  
    /**  
        body of the method  
    */  
    return 0;  
}
```

Using Methods

Methods are helpful for multiple reasons:

Allows us to use functionality without knowing how it works

Example: `Math.random()`

Allows us to encapsulate reusable code, so we don't have to copy and paste it over and over.

Creating Methods

Idea: Define re-useable portions of code

Analogy: machines with inputs and outputs

Two steps for programming with functions:

1. Define the function (name, inputs, outputs, implementation)
2. Call the function with inputs and do something with its output

All methods should be contained inside a class (more on classes later)

Method Signature

```
public static int method1 (int param1, String param2)
```

Method Documentation

```
/**
Description of the method
* @param param1 description
* @param param2 description
* @return what the method returns
*/
public static int method1 (int param1,
                           String param2) {
    // ...
}
```

Documentation is Important!

Helps other programmers understand the functionality, parameters, and return type of your function.

Helps **you** remember what your code does.

Style

- How we format our programs is **very** important
 - Like rules of etiquette around eating and keep a clean appearance
 - Like punctuation rules, it helps make text more readable
- Variable names should be descriptive
- Indentation is **very** important
 - Every statement inside a pair of braces must be indented
- Braces should be placed consistently
- Follow camel case (ex: isOdd, sumTwoNumbers)

Defining Methods in Java: Syntax

```
public static void main(String[] args) {  
    // function statements  
}
```

```
public static float foo(int a, float b, String c) {  
    // function statements  
    System.out.println(c);  
    return a*b;  
}
```

Calling Methods in Java: Syntax

```
public static float foo(int a, float b, String c) {  
    // function statements  
    System.out.println(c);  
    return a*b;  
}
```

parameters

```
public static void main(String[] args) {  
    // function statements  
    int value = 3;  
    String c = "hello";  
    float result = foo(value, -2.5, c);  
    System.out.println(result);  
}
```

arguments

Executing a method: steps

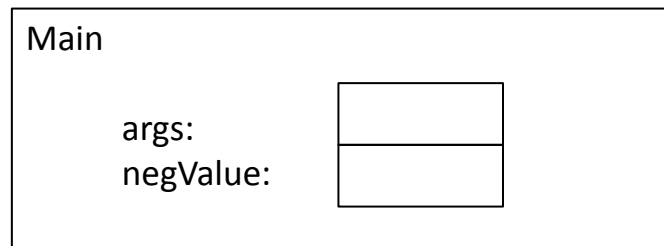
1. When you encounter a method, pause!
2. Create a *frame* to hold the method state
3. Copy argument values
4. Execute the method, line by line. Continue until
 1. you hit a return statement
 2. you run out of statements
5. Send back return value (can be nothing if function is *void*)
6. Delete the method's frame
7. Resume original function

Exercise: Draw stack diagram

```
public class Neg {  
  
    public static double neg(double x) {  
        double value = x * -1  
        return value;  
    }  
  
    public static void main(String[] args) {  
        double negValue = 0;  
        negValue = neg(-3.4);  
    }  
}
```

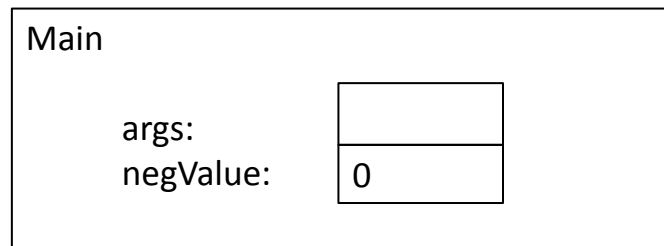
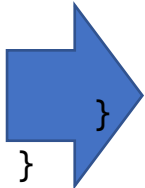
Exercise: Draw stack diagram

```
public class Neg {  
  
    public static double neg(double x) {  
        double value = x * -1;  
        return value;  
    }  
  
    public static void main(String[] args) {  
        double negValue = 0;  
        negValue = neg(-3.4);  
    }  
}
```



Exercise: Draw stack diagram

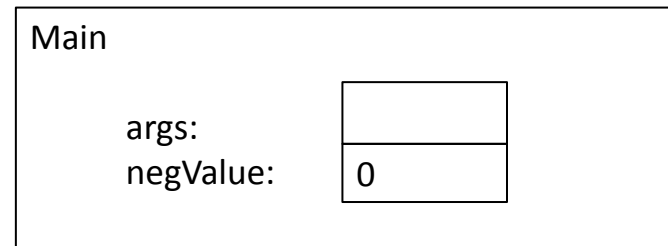
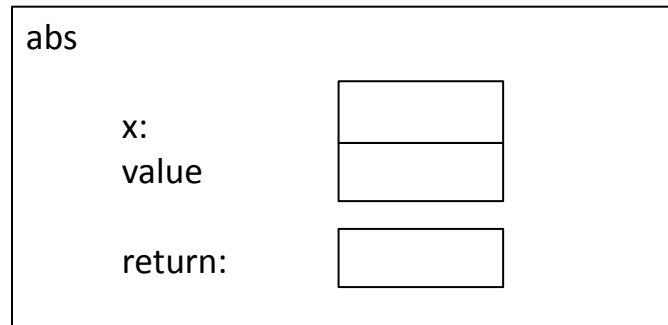
```
public class Neg {  
  
    public static double neg(double x) {  
        double value = x * -1;  
        return value;  
    }  
  
    public static void main(String[] args) {  
        double negValue = 0;  
        negValue = neg(-3.4);  
    }  
}
```



Exercise: Draw stack diagram



```
public class Neg {  
  
    public static double neg(double x) {  
        double value = x * -1;  
        return value;  
    }  
  
    public static void main(String[] args) {  
        double negValue = 0;  
        negValue = neg(-3.4);  
    }  
}
```



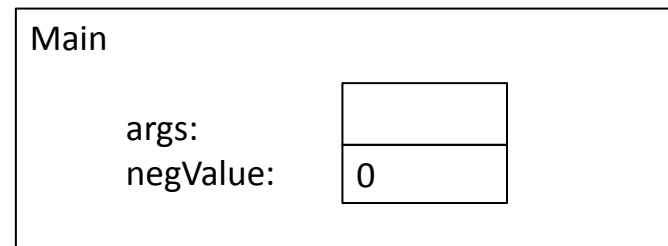
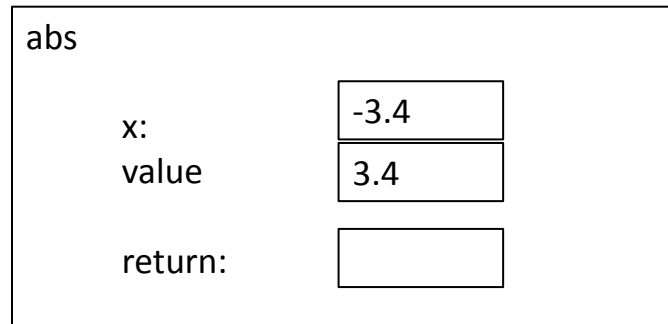
Exercise: Draw stack diagram

```
public class Neg {
```



```
    public static double neg(double x) {  
        double value = x * -1;  
        return value;  
    }
```

```
    public static void main(String[] args) {  
        double negValue = 0;  
        negValue = neg(-3.4);  
    }  
}
```



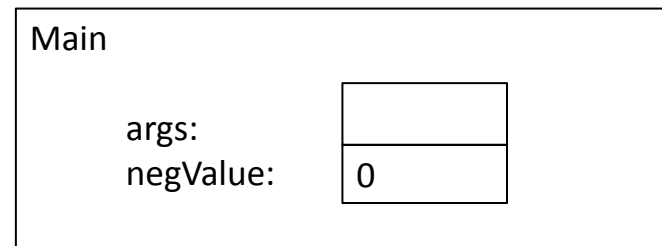
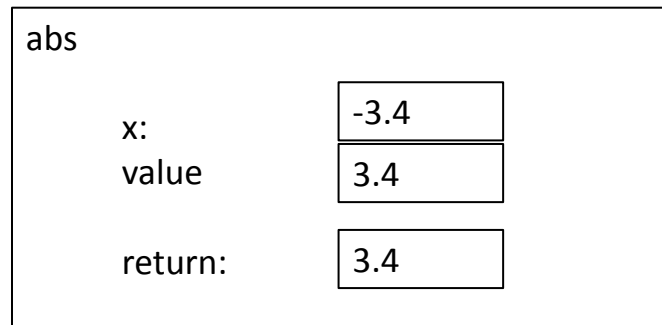
Exercise: Draw stack diagram

```
public class Neg {
```



```
    public static double neg(double x) {  
        double value = x * -1;  
        return value;  
    }
```

```
    public static void main(String[] args) {  
        double negValue = 0;  
        negValue = neg(-3.4);  
    }  
}
```



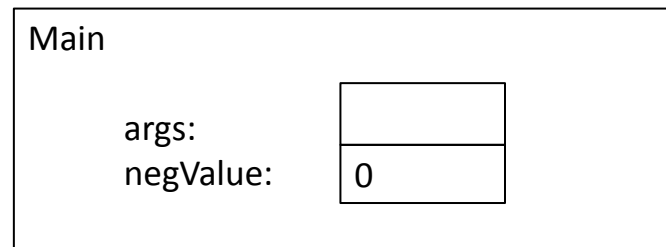
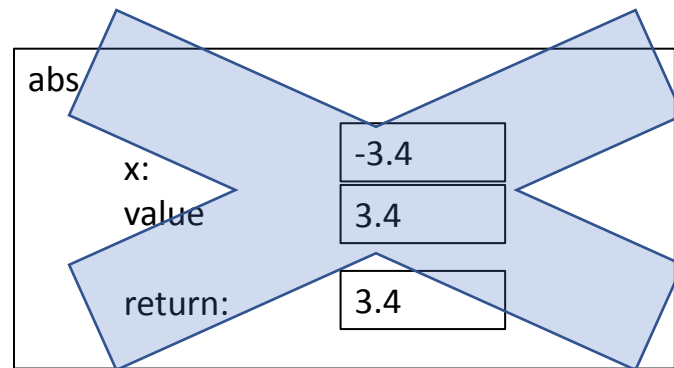
Exercise: Draw stack diagram

```
public class Neg {
```

```
    public static double neg(double x) {  
        double value = x * -1  
        return value;  
    }
```

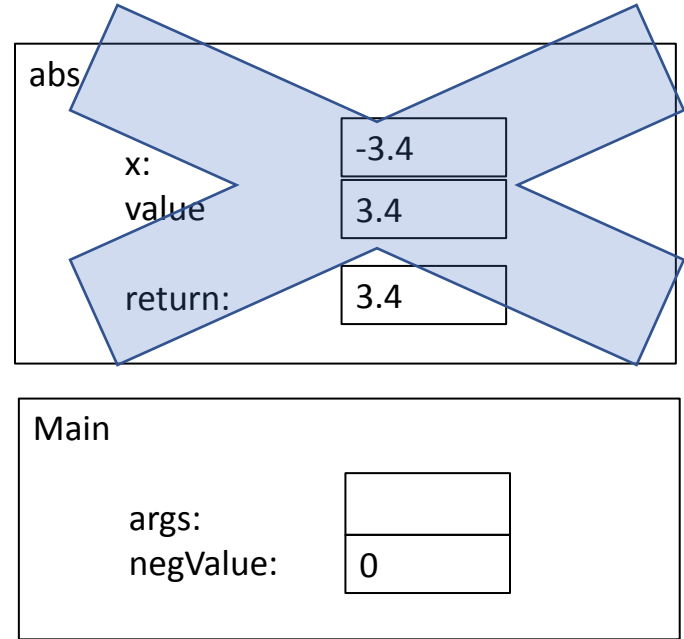


```
    public static void main(String[] args) {  
        double negValue = 0;  
        negValue = neg(-3.4);  
    }  
}
```



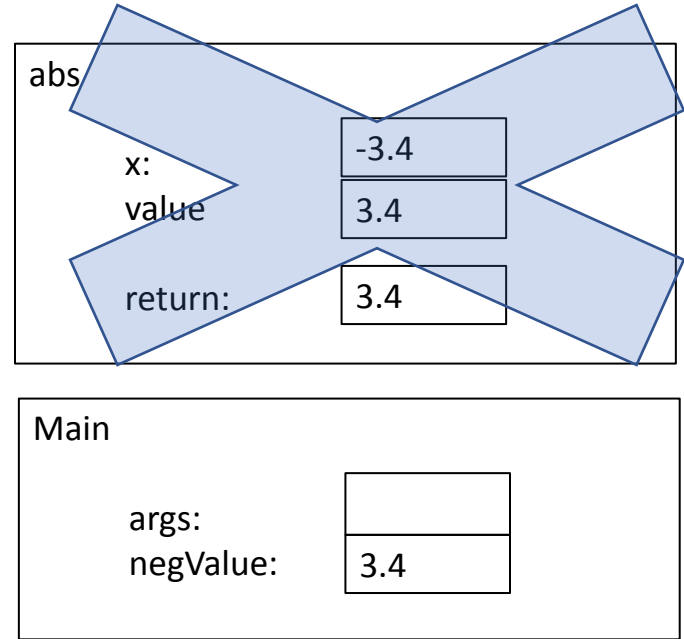
Exercise: Draw stack diagram

```
public class Neg {  
  
    public static double neg(double x) {  
        double value = x * -1;  
        return value;  
    }  
  
    public static void main(String[] args) {  
        double negValue = 0;  
        negValue = neg(-3.4);  
    }  
}
```



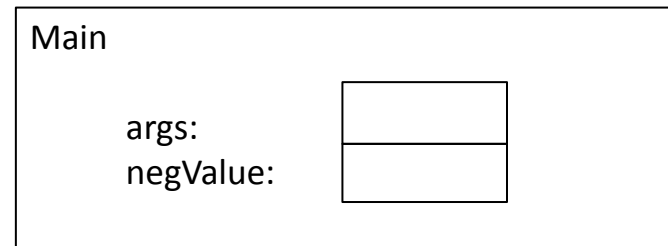
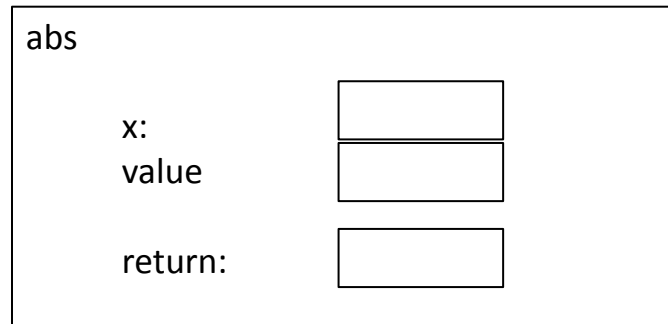
Exercise: Draw stack diagram

```
public class Neg {  
  
    public static double neg(double x) {  
        double value = x * -1;  
        return value;  
    }  
  
    public static void main(String[] args) {  
        double negValue = 0;  
        negValue = neg(-3.4);  
    }  
}
```



Exercise: Draw stack diagram

```
public class Neg {  
  
    public static double neg(double x) {  
        double value = x * -1;  
        return value;  
    }  
  
    public static void main(String[] args) {  
        double negValue = 0;  
        negValue = neg(5.4);  
    }  
}
```



Exercise: Draw stack diagram

```
public class Abs {  
  
    public static double abs(double x) {  
        if (x < 0) {  
            return -x;  
        }  
        return x;  
    }  
  
    public static void main(String[] args) {  
        double absValue = 0;  
        absValue = abs(5.4);  
    }  
}
```

What is different here?

```
// Function: area
// Description: computes the area of a rectangle
// Input: width (double)
// Input: height (double)
// returns (double), the area as width * height
// side effects: none
public static double area(double width, double height) {
    return width * height;
}
```

```
// Function: area
// Description: computes the area of a rectangle
// Input: width (double)
// Input: height (double)
// returns (none)
// Side effect: prints the area to the console
public static void area(double width, double height)
{
    double a = width * height;
    System.out.println("Area is " + a);
}
```

Warning: don't confuse printing with returning

```
// Function: area
// Description: computes the area of a rectangle
// Input: width (double)
// Input: height (double)
// returns (double), the area as width * height
// side effects: none
public static double area(double width, double height) {
    return width * height;
}
```

```
// Function: area
// Description: computes the area of a rectangle
// Input: width (double)
// Input: height (double)
// returns (none)
// Side effect: prints the area to the console
public static void area(double width, double height)
{
    double a = width * height;
    System.out.println("Area is " + a);
}
```

Benefits of methods

- Split large problems into small problems
- Easier to maintain code/cleaner code
 - Only need to fix mistakes
 - DRY: Don't repeat yourself
- Implement once, re-use in different programs
- Abstract details so user doesn't need to worry about details

Method specifications

Idea: “contract” between the function user and the method implementation

- Inputs and their types

- Return type

- Description of how function behaves, including special cases and side effects

A **side effect** refers to changes the method makes that last after the method returns (e.g. printing to the console is a side effect)

The **method signature** includes just the inputs and outputs of the function

Method Specifications

```
/**  
 * Takes two integers and returns their sum.  
 *  
 * @param firstInt  
 * @param secondInt  
 * @ returns the sum of firstInt and secondInt  
 * /  
public static int sumTwo(int firstInt, int secondInt) {  
    return firstInt + secondInt;  
}
```

Why have method specifications?

- Make the behavior of function clear
- Enable user to use function without having to look at the implementation

Testing

How do we know our methods are implemented correctly? → testing!

Testing is calling the method with different inputs and checking the results

Unit testing: test a small unit of code

- Java has many built in tools for this, we will cover them later in the semester

For now: MVT (minimum viable testing)

- Call the method you want to test from the main() function with various inputs
- Think of *edge cases* (ex: reading an integer in from a user → what if they pass in a float instead? How does your method handle that?)

Top down design

1. Identify features of the program
 1. List them out!
2. Identify verbs and nouns in feature list
 1. Verbs: functions
 2. Nouns: objects/variables
3. Sketch major steps – how features should fit together
 1. Algorithm!
4. Write program skeleton
 1. Include function **stubs** (placeholders for our functions)
 2. Function **stub**: empty function with parameters and return type
5. Implement and test function stubs one at a time

Let's try it out

Let's use top down design to write a function: `isVowel()`, and test it by calling it from the `main()` function